

Top Maşını

Mədinə IOI iştirakçılarını əyləndirmək üçün top maşını icad edib. Maşının daxili quruluşu aşağıdakı kimidir:

- Maşın, 0 -dan $N - 1$ -a qədər indekslənmiş N **qovşaqdan** ibarətdir. $N - 1$ qovşağı **kök** adlanır.
- $0 \leq u < N - 1$ olan hər bir u qovşağının **valideyni** $P[u]$ qovşağıdır, $P[u] > u$ və u qovşağı $P[u]$ -un **uşaq** qovşağıdır. Kökün valideyni yoxdur. Uşaqsız qovşağa **yarpaq** deyilir.

Siz nə N nə də istənilən u qovşağının valideyni olan $P[u]$ qovşağını bilmirsiniz. Bunun əvəzinə, Mədinə sizə yarpaqların sayının M olduğunu və yarpaqların indekslərinin $0, 1, \dots, M - 1$ olduğunu deyir. Sizin tapşırığınız maşını işlədərək onun daxili quruluşunu təyin etməkdir.

Maşının hər qovşağı ən çoxu bir **top** saxlaya bilər və hər bir topun mənfi olmayan tam ədəd olan bir **dəyəri** var. Əgər qovşaq dəyəri x olan bir topu saxlayırsa, biz deyirik ki, qovşaq x dəyərində malikdir. Əgər qovşaqda top varsa, o, **dolu** sayılır; əks halda **boş**. Əvvəlcə hər qovşaq boşdur.

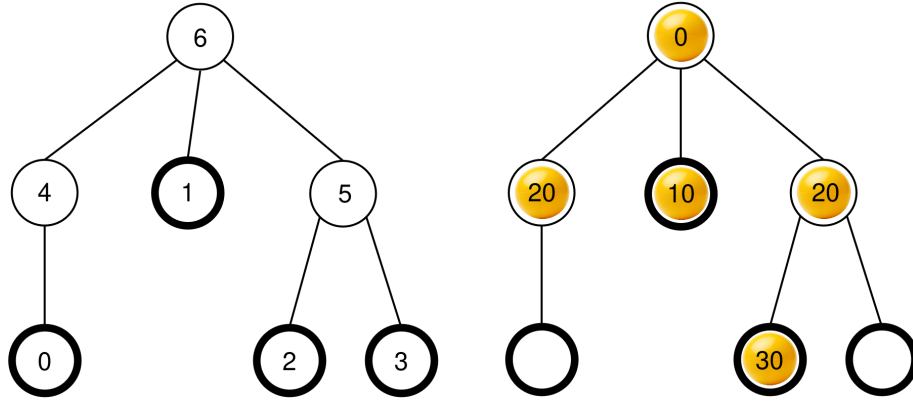
İki növ əməliyyat edə bilərsiniz:

- insert** (U, X) : U yarpağına X dəyəri olan yeni bir top daxil etməyə cəhd.
 - Əgər U yarpağı doludursa, əməliyyat topu daxil etmədən `false` qaytarır.
 - Əgər U yarpağı boşdursa, top U qovşağına yerləşdirilir. Daha sonra, topun olduğu qovşağın valideyni boş olduğu müddətcə, top cari qovşağın valideyninə doğru dayanmadan hərəkət edir. Hərəkət, top kökə və ya valideyni dolu olan qovşağa çatdıqda dayanır. Əməliyyat `true` qaytarır.
- collect()**: əgər kök boşdursa (yəni maşın boşdursa), `collect` boş bir massiv qaytarır. Əks halda, aşağıda təsvir edilən `traverse` rekursiv proseduru hazırda maşında olan bütün topların dəyərlərini S massivinə toplayır. Əvvəlcə S massivi boşdur və `traverse(N - 1)` çağırılır.

```
traverse(u) :  
    u qovşağındakı topun dəyərini S-in sonuna əlavə edin  
    c[u] u-nun dolu olan uşaqlarının siyahısı olsun  
    c[u]-nu qovşaqlarda olan topların dəyərlərinə görə azalmayan qaydada sıralayın  
    (əgər bir neçə qovşaq eyni dəyərdədirsə,  
        onlar öz aralarında istənilən sıra ilə düzülə bilər)  
    for each v in c[u]:  
        traverse(v)
```

Bu prosedur başa çatdıqdan sonra, **bütün toplar maşından çıxarılır** və `collect` S qaytarır.

Misal üçün, $N = 7$ sayda qovşaq və $P = [4, 6, 5, 5, 6, 6]$ valideyn massivi olan bir maşını nəzərdən keçirin. Soldakı şəkildə qovşaq indeksləri olan boş maşını, sağdakı şəkildə isə müəyyən bir `insert` əməliyyatları ardıcılığından sonra topların mümkün konfigurasiyasını göstərir (bu ardıcılıq Nümunə bölməsində verilmişdir).



`collect()` çağırıldıqda, kök (6 qovşağı) dolu olur, beləliklə, $S = []$ kimi başlayır və `traverse(6)` çağırılır.

traverse(6) : 6 qovşağının dəyəri 0 -dir; onu S -ə əlavə etdikdə $S = [0]$ alınır. 6 qovşağının uşaqları 4 , 1 və 5 -dir və hamısı doludur. Onların dəyərləri müvafiq olaraq 20 , 10 və 20 -dir. Azalmayan qaydada düzəldə $c[6] = [1, 5, 4]$ verir (qeyd edək ki, 4 və 5 qovşaqları eyni dəyərə malik olduğundan $c[6] = [1, 4, 5]$ də düzgün olardı). Daha sonra prosedur $c[6]$ -dakı hər bir qovşaq üçün `traverse` prosedurunu həmin qaydada çağırır.

- **traverse(1)** : 1 qovşağının dəyəri 10 -dur; onu S -ə əlavə etdikdə $S = [0, 10]$ alınır. 1 qovşağının dolu uşağı yoxdur, buna görə də bu çağırış başa çatır.
- **traverse(5)** : 5 qovşağının dəyəri 20 -dir; onu S -ə əlavə etdikdə $S = [0, 10, 20]$ alınır. 5 qovşağının bir dolu uşağı var, 2 qovşağı , buna görə də $c[5] = [2]$ və prosedur `traverse(2)` çağırır.
 - **traverse(2)** : 2 qovşağının dəyəri 30 -dur; onu S -ə əlavə etdikdə $S = [0, 10, 20, 30]$ alınır. 2 qovşağının dolu uşaqları yoxdur, buna görə də bu çağırış başa çatır.
 - İcra $c[5]$ dəki bütün uşaqları emal edən `traverse(5)` funksiyasına qayıdır və beləliklə, onun çağırışı başa çatır.
- **traverse(4)** : 4 qovşağının dəyəri 20 -dir; onu S -ə əlavə etdikdə $S = [0, 10, 20, 30, 20]$ alınır. 4 qovşağının dolu uşaqları yoxdur, buna görə də bu çağırış başa çatır.

Bütün çağırışlar başa çatdı və emal ediləcək başqa qovşaq qalmadı. Nəhayət, hər bir top maşından çıxarılır və `collect` $S = [0, 10, 20, 30, 20]$ massivini qaytarır.

Tapşırığınız qovşaqların sayı olan N ədədini təyin etmək və maşının strukturunu təsvir edən, lakin yarpaq və kök olmayan qovşaqların potensial olaraq fərqli nömrələnməsi ilə valideyn $R = [R[0], R[1], \dots, R[N - 2]]$ massivini bildirməkdir. Formal olaraq, bildirilən R massivi, maşının hər bir u qovşağına fərqli $L[u]$ ($0 \leq L[u] < N$) nömrəsi təyin etmək mümkün olduqda düzgün hesab olunur:

- Hər bir $0 \leq u < M$ və $u = N - 1$ üçün $L[u] = u$
- Hər bir $0 \leq u < N - 1$ üçün $L[P[u]] = R[L[u]]$

$R[u] > u$ olması tələb olunmur. Həllinizi bildirərkən maşın **boş** olmalıdır.

Tutaq ki, K yerinə yetirilən `collect` əməliyyatlarının sayı, B isə bütün `insert` çağırışları üzrə topun maksimum dəyəridir. Tutaq ki $C = K + B$. Onda C 1000 **ədədini keçməməlidir**. Bəzi alt tapşırıqlardakı balınız C -in dəyərindən asılıdır.

İcra Detalları

Aşağıdakı proseduru icra etməlisiniz.

```
std::vector<int> find_structure(int M)
```

- M : maşındakı yarpaqların sayı.
- Prosedur hər test üçün tam olaraq bir dəfə çağırılır.
- Prosedur, uzunluğu $N - 1$ olan, düzgün valideyn massivi olan $R = [R[0], R[1], \dots, R[N - 2]]$ massivi qaytarmalıdır.

Maşınla qarşılıqlı əlaqə qurmaq üçün prosedurunuz aşağıdakı iki proseduru çağırma bilər.

Birinci prosedur belədir:

```
bool insert(int U, int X)
```

- U : topun daxil edilməli olduğu yarpağın indeksi. $0 \leq U < M$ şərti ödənməlidir.
- X : topun tam ədəd olan dəyəri. $0 \leq X \leq 1000$ şərti ödənməlidir.
- Bu prosedur, top uğurla daxil edildikdə `true`, U yarpağı artıq dolu olduqda isə `false` qaytarır.
- Bu prosedur maksimum 500 000 dəfə çağırıla bilər.

İkinci prosedur belədir:

```
std::vector<int> collect()
```

- Daxil edilmiş bütün topların dəyərlərini yığır, maşını boşaldır və S massivini qaytarır.

Proqramınızın icrası zamanı hər hansı bir anda C dəyəri 1000 -i keçərsə, həlliniz `Output isn't correct: Too many resources used` nəticəsini alacaq.

Maşının strukturu `find_structure` çağırılmazdan əvvəl sabitdir. Qiymətləndirici **deterministikdir**, onu iki dəfə işə salsanız və hər səfər eyni əməliyyat ardıcılığı yerinə yetirilsə, `collect` çağırışları eyni massivləri qaytaracaq.

Məhdudiyyətlər

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Alt tapşırıqlar

Alt tapşırıq	Bal	Əlavə Məhdudiyyətlər
1	5	Kökün tam olaraq M uşağı var.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Əlavə məhdudiyyət yoxdur

3-cü və 4-cü alt tapşırıqlarda balınız aşağıdakı kimi C -in dəyərindən asılıdır.

Alt tapşırıq 3 (25 bal)

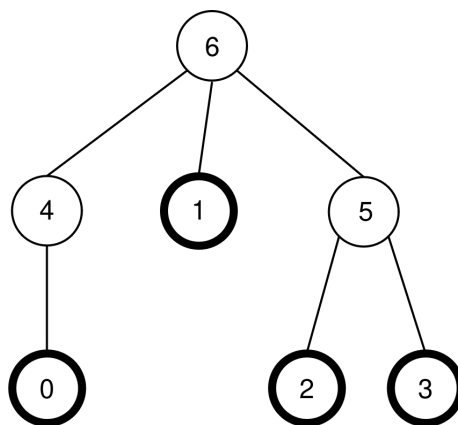
Limitlər	Bal
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Alt tapşırıq 4 (60 bal)

Limitlər	Bal
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Nümunə

Tapşırıq təsvirindəki ilə eyni maşını nəzərdən keçirin, yəni $N = 7$, $M = 4$ və $P = [4, 6, 5, 5, 6, 6]$. Maşın yenidən aşağıdakı şəkildə göstərilib:



Qiymətləndirici aşağıdakı çağırışı edir:

```
find_structure(4)
```

Prosedur aşağıdakı çağırış ardıcılığını edə bilər:

1. **insert(0, 0)** : 0 yarpağı boşdur, ona görə də 0 dəyəri olan top 0 yarpaqda yerləşdirilir. $P[0] = 4$ qovşağı boşdur, ona görə də top 4 qovşağına keçir. $P[4] = 6$ qovşağı boşdur, ona görə də top 6 qovşağına keçir. 6 qovşağı kök olduğundan, hərəkət dayanır. Çağırış `true` qaytarır.
2. **insert(3, 20)** : 3 yarpağı boşdur, ona görə də 20 dəyərində olan top 3 yarpağına yerləşdirilir. $P[3] = 5$ qovşağı boşdur, ona görə də top 5 qovşağına keçir. $P[5] = 6$ boş olduğundan, hərəkət dayanır. Çağırış `true` qaytarır.
3. **insert(1, 10)** : 1 yarpağı boşdur, ona görə də 10 dəyərində olan top 1 yarpağına yerləşdirilir. $P[1] = 6$ dolu olduğundan, top 1 qovşağında qalır. Çağırış `true` qaytarır.
4. **insert(2, 30)** : 2 yarpağı boşdur, ona görə də 30 dəyərində olan top 2 yarpağına yerləşdirilir. $P[2] = 5$ dolu olduğundan, top 2 qovşağında qalır. Çağırış `true` qaytarır.
5. **insert(1, 25)** : 1 yarpağı yeri doludur, ona görə də top 1 yarpaq yerinə qoyulmur. Çağırış `false` qaytarır.
6. **insert(0, 20)** : 0 yarpağı boşdur, ona görə də 20 dəyərində olan top 0 yarpağına yerləşdirilir. $P[0] = 4$ qovşağı boşdur, ona görə də top 4 qovşağına keçir. $P[4] = 6$ dolu olduğundan, hərəkət dayanır. Çağırış `true` qaytarır.

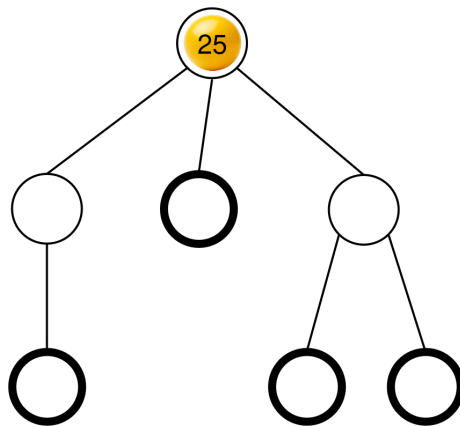
Topların nəticədə əldə edilən konfigurasiyası artıq tapşırıq təsvirində göstərilmişdi.

Bundan sonra prosedur aşağıdakıları çağırma bilər:

```
collect()
```

Bu çağırışın icrası artıq tapşırıq təsvirində izah edilmişdi və çağırış $S = [0, 10, 20, 30, 20]$ qaytarır. Daha sonra maşın yenidən boş olur.

Daha sonra prosedur **insert(2, 25)** çağırma bilər. 2 yarpağı boşdur, ona görə də 25 dəyəri olan top 2 yarpağına yerləşdirilir. Bu top daha sonra 5 qovşağına, daha sonra isə 6 qovşağına keçir və orada dayanır. Çağırış `true` qaytarır. Nəticədə topaların konfigurasiyası aşağıdakı şəkildə göstərilir.



Nəhayət, prosedur **collect()** funksiyasını çağırma bilər ki, bu da $S = [25]$ qaytarır və maşını boşaldır.

`find_structure` qaytara biləcəyi iki düzgün massiv var: $R = P = [4, 6, 5, 5, 6, 6]$ ($L = [0, 1, 2, 3, 4, 5, 6]$ nömrələnməsinə uyğundur) və $R = [5, 6, 4, 4, 6, 6]$ ($L = [0, 1, 2, 3, 5, 4, 6]$ nömrələnməsinə uyğundur). Bu nümunədə, $K = 2$ və $B = 30$, buna görə də $C = 32$.

Nümunə Qiymətləndirici

Giriş formatı:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Çıxış formatı:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Burada, Q `find_structure` tərəfindən qaytarılan R massivinin uzunluğudur.